

CO3 ASSESSMENT TOOL – 1
DESIGN AND ANALYSIS OF ALGORITHMS

Experiment Report

**Evaluate the Effectiveness of Merge Sort in External
Sorting Scenarios**

by Simulating Disk-Based Datasets

Name	Udhayakumar U
Register Number	192424332
Course Code	CSA-0606
Course	Design And Analysis Algorithm

1. Assessment Question

Evaluate the effectiveness of Merge Sort in external sorting scenarios by simulating disk-based datasets. Record execution time and I/O operations. Analyze how divide and conquer helps manage large data that cannot fit in memory. Interpret the results and justify its use in real-world large-scale systems.

Problem Statement

Traditional in-memory sorting assumes that the complete dataset can fit into RAM. When the dataset is larger than available memory, data must be stored on secondary storage and processed in manageable portions. This experiment simulates such an external sorting environment using files and evaluates Merge Sort by measuring execution time and simulated disk I/O operations.

Input Requirements

A set of integer records stored in a disk-based input file. The experiment uses different dataset sizes to represent data that can be processed in chunks.

Expected Output

A sorted output file containing all input records in ascending order, together with execution time and the number of simulated read/write I/O operations.

Constraints

- The complete dataset is assumed not to fit in main memory.
- Only a fixed-size chunk is loaded into memory at a time.
- Temporary files are used during the merge process.
- The input records are integers for simple measurement and verification.

Assessment Criteria

Correct implementation of external Merge Sort; correct handling of disk-based data; accurate measurement of execution time and I/O operations; complexity analysis; interpretation of results; and justification of the algorithm for large-scale systems.

2. Objective

To implement and evaluate an external Merge Sort algorithm using disk-based datasets, measure execution time and simulated I/O operations, and analyze how the divide-and-conquer strategy enables sorting of datasets larger than available main memory.

3. Required Resources

Hardware Requirements

- Computer/Laptop with at least 4 GB RAM.
- Adequate secondary storage for input and temporary files.

Software / Tools

- Python 3.x
- Operating system: Windows/Linux/macOS
- Command Prompt/Terminal
- VS Code or another Python-compatible IDE

Programming Language

- Python

IDE / Compiler

- Visual Studio Code with Python extension, or Python IDLE

Dataset / Input Files

- Generated integer datasets of different sizes.
- Temporary chunk files generated by the program during external sorting.

4. Algorithm / Procedure

- 1. Generate or obtain a large input dataset containing integer records.
- 2. Define a fixed memory chunk size so that only a limited number of records are loaded at once.
- 3. Read one chunk from the input file and count the read operation.
- 4. Sort the chunk in memory using Merge Sort.
- 5. Write the sorted chunk to a temporary file and count the write operation.
- 6. Repeat the process until all input records have been converted into sorted temporary runs.
- 7. Open the temporary sorted runs and merge them progressively using a k-way merge.
- 8. Count file reads and writes during the merge phase.
- 9. Write the final sorted sequence to the output file.
- 10. Measure the total execution time using a high-resolution timer.
- 11. Verify that the final output is sorted correctly.
- 12. Compare execution time and I/O operations across different dataset sizes.

How Divide and Conquer Helps

External Merge Sort divides a large dataset into smaller chunks that fit in memory. Each chunk is sorted independently, creating sorted runs on disk. The sorted runs are then merged to produce the final sorted file. Thus, the algorithm avoids loading the entire dataset into RAM while retaining the predictable $O(n \log n)$ comparison complexity.

5. Implementation

The following Python program simulates external sorting. It generates disk-based data, creates sorted runs using Merge Sort, merges the runs, counts simulated I/O operations, records execution time, and verifies the final output.

```
import os
import random
import heapq
import time
import tempfile

IO_READS = 0
IO_WRITES = 0

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0
```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

```

```

result.extend(left[i:])
result.extend(right[j:])
return result

```

```

def generate_input(filename, n):
    global IO_WRITES
    with open(filename, "w") as f:
        for _ in range(n):
            f.write(str(random.randint(1, 10**9)) + "\n")
            IO_WRITES += 1

```

```

def external_merge_sort(input_file, output_file, chunk_size):
    global IO_READS, IO_WRITES
    IO_READS = 0
    IO_WRITES = 0
    temp_files = []

```

```

# Phase 1: Create sorted runs
with open(input_file, "r") as f:
    while True:
        chunk = []
        for _ in range(chunk_size):
            line = f.readline()
            if not line:

```

```

        break
    IO_READS += 1
    chunk.append(int(line.strip()))

if not chunk:
    break

sorted_chunk = merge_sort(chunk)

temp = tempfile.NamedTemporaryFile(
    mode="w", delete=False, suffix=".tmp"
)
for value in sorted_chunk:
    temp.write(str(value) + "\n")
    IO_WRITES += 1
temp.close()
temp_files.append(temp.name)

# Phase 2: Merge sorted runs
files = [open(name, "r") for name in temp_files]
heap = []

for idx, f in enumerate(files):
    line = f.readline()
    if line:
        IO_READS += 1
        heapq.heappush(heap, (int(line.strip()), idx))

with open(output_file, "w") as out:
    while heap:
        value, idx = heapq.heappop(heap)
        out.write(str(value) + "\n")

```

```
IO_WRITES += 1
```

```
line = files[idx].readline()
```

```
if line:
```

```
    IO_READS += 1
```

```
    heapq.heappush(heap, (int(line.strip()), idx))
```

```
for f in files:
```

```
    f.close()
```

```
for name in temp_files:
```

```
    os.remove(name)
```

```
def verify_sorted(filename):
```

```
    previous = None
```

```
    with open(filename, "r") as f:
```

```
        for line in f:
```

```
            value = int(line.strip())
```

```
            if previous is not None and value < previous:
```

```
                return False
```

```
            previous = value
```

```
    return True
```

```
def run_experiment(n, chunk_size):
```

```
    input_file = "external_input.txt"
```

```
    output_file = "external_sorted.txt"
```

```
    generate_input(input_file, n)
```

```
    start = time.perf_counter()
```

```
    external_merge_sort(input_file, output_file, chunk_size)
```

```
    elapsed = time.perf_counter() - start
```



```

correct = verify_sorted(output_file)

result = (n, chunk_size, elapsed, IO_READS, IO_WRITES, correct)

os.remove(input_file)
os.remove(output_file)

return result

if __name__ == "__main__":
    print("External Merge Sort Evaluation")
    print("-" * 72)
    print("Records | Chunk | Time(s) | Reads | Writes | Correct")

    for n in [10000, 50000, 100000]:
        result = run_experiment(n, 1000)
        print(
            f"{result[0]:7d} | {result[1]:5d} | "
            f"{result[2]:7.4f} | {result[3]:5d} | "
            f"{result[4]:6d} | {result[5]}"
        )

```

Important Functions / Modules

Function / Module	Purpose
<code>merge_sort()</code>	Performs recursive divide-and-conquer sorting on each memory-resident chunk.
<code>generate_input()</code>	Creates the disk-based integer dataset.
<code>external_merge_sort()</code>	Creates sorted runs and merges them to form the final sorted output.

verify_sorted()	Checks whether the final output file is in non-decreasing order.
run_experiment()	Runs one dataset-size experiment and records performance measurements.
time.perf_counter()	Provides high-resolution execution-time measurement.
heapq	Supports efficient k-way merging of sorted runs.
tempfile	Creates temporary files used as disk-based sorted runs.

6. Input and Output

The experiment uses generated integer records. The chunk size is fixed at 1,000 records to simulate a restricted-memory environment. The following test cases can be executed using the supplied program.

Test Case	Input Records	Chunk Size	Expected Output	Verification
1	10,000 random integers	1,000	Ascending sorted file	True
2	50,000 random integers	1,000	Ascending sorted file	True
3	100,000 random integers	1,000	Ascending sorted file	True

Example console output format:

External		Merge			Sort		Evaluation			

Records		Chunk		Time(s)		Reads		Writes		Correct
10000		1000		0.xxxx		...		...		True
50000		1000		0.xxxx		...		...		True
100000		1000		0.xxxx		...		...		True

Note: Exact execution times and I/O counts depend on the computer, operating system, storage device, Python version, and system load. The program should be executed to record the actual measured values rather than using fixed values.

7. Results

Record the actual measurements produced by the program in the following table after execution.

Dataset Size	Chunk Size	Execution Time (s)	I/O Reads	I/O Writes	Sorted Correctly
10,000	1,000	To be measured	To be measured	To be measured	Yes
50,000	1,000	To be measured	To be measured	To be measured	Yes
100,000	1,000	To be measured	To be measured	To be measured	Yes

Expected observation: as the number of records increases, the execution time and the number of file-level I/O operations generally increase. The exact relationship depends on chunk size, storage performance, operating-system caching, and the number of sorted runs.

Suggested graph: Dataset Size vs. Execution Time; Dataset Size vs. Total I/O Operations.

8. Analysis of Results

Expected Output Analysis

The verification step should return True for every test case. This confirms that the final output is correctly sorted even though the complete dataset was not held in memory.

Performance Analysis

External Merge Sort performs sequential reads and writes and divides the work into memory-sized runs. For large files, sequential disk access is generally preferable to random access. The number of temporary runs decreases as the chunk size increases, provided sufficient memory is available.

Time Complexity

The comparison-based sorting complexity is $O(n \log n)$. In external sorting, the practical cost also depends heavily on the number of passes over the data and the I/O cost of the storage device.

Space Complexity

The in-memory working space is $O(M)$, where M is the chosen chunk size. Additional disk space is required for temporary sorted runs and the final output, so total external storage usage is $O(n)$.

Advantages

- Can sort datasets larger than available RAM.
- Uses predictable divide-and-conquer processing.
- Merge phase can use sequential disk access efficiently.
- Stable Merge Sort can preserve the order of equal records.
- Well suited to database systems, data warehouses, log processing, and big-data pipelines.

Limitations

- Disk I/O can be much slower than memory operations.
- Temporary files require additional storage.
- Performance depends strongly on storage hardware and I/O bandwidth.
- The number of merge passes can increase when memory/chunk size is small.

Comparison with Alternative Approaches

Approach	Memory Requirement	Large Dataset Support	Suitability for External Sorting
In-memory Merge Sort	$O(n)$	Limited by RAM	Low when n exceeds RAM
Quick Sort	$O(\log n)$ average stack	Needs data accessible in memory	Less suitable for disk-based sorting
External Merge Sort	$O(M)$ working memory	Yes	Excellent

Real-World Justification

External Merge Sort is widely applicable when data volumes exceed memory capacity. Database management systems use external sorting for operations such as ORDER BY, GROUP BY, duplicate elimination, merge joins, and index-related processing.

9. Conclusion

The experiment demonstrates that External Merge Sort is effective for sorting datasets that cannot fit entirely in main memory. Divide and conquer breaks the dataset into manageable memory-sized chunks, sorts each chunk, and then merges the sorted runs from disk. Execution time and I/O operations provide practical evidence of the cost of external sorting. Although disk I/O introduces overhead compared with purely in-memory sorting, External Merge Sort provides a scalable and reliable solution for large-scale data processing.